Tarea: Aprendizaje no Supervisado

Ciencia de Datos — Universidad Católica del Norte

Objetivos de esta tarea

- Implementar k-means completo: asignación, actualización, criterio de parada e inercia, y constatar que el resultado depende de dónde partieron los centroides.
- Sembrar los centroides con k-means++ y medir cuánto mejora respecto del sorteo uniforme.
- Evaluar un agrupamiento sin usar los nombres de los grupos, con el índice de Rand entre dos particiones.
- Calcular PCA desde la matriz de covarianza y verificarlo con una reconstrucción de rango bajo, comprobando que lo que se pierde es exactamente la varianza descartada.
- Agrupar por densidad con DBSCAN, distinguir núcleo, borde y ruido, y reconocer en qué conjuntos falla cada método.

Contexto

Todo lo que vimos hasta ahora partía de un conjunto de pares $(\mathbf{x}_i, y_i)$: alguien ya había decidido cuál era la respuesta correcta para cada observación. Esta tarea trabaja sin esa columna. Solo hay una matriz X de n filas y d columnas, y la pregunta es qué estructura tiene: si las filas se juntan en grupos, y si las d columnas son realmente d direcciones distintas de variación o en el fondo son menos.

Las dos preguntas se abordan con dos familias de métodos, y esta tarea implementa las cuatro piezas centrales: **k-means** y **DBSCAN** para agrupar, **PCA** para reducir dimensiones y el **índice de Rand** para poder decir, con un número, si un agrupamiento se parece a otro.

k-means

k-means busca k centroides $\mathbf{c}_1, \ldots, \mathbf{c}_k$ que minimicen la **inercia**, la suma de distancias al cuadrado de cada punto al centroide que tiene asignado:

$$J(\mathbf{c}_1, \ldots, \mathbf{c}_k) \;=\; \sum_{i=1}^{n} \left\|\mathbf{x}_i - \mathbf{c}_{a(i)}\right\|^2, \qquad a(i) = \arg\min_{j} \left\|\mathbf{x}_i - \mathbf{c}_j\right\|^2.$$

Encontrar el mínimo global es un problema duro, así que el algoritmo alterna dos pasos, cada uno de los cuales resuelve exactamente una mitad del problema dejando la otra fija:

Algoritmo de Lloyd (k-means). Partir de k centroides iniciales. Repetir:

1. **Asignar**: cada punto al centroide más cercano (con los centroides fijos, es la mejor asignación posible).
2. **Actualizar**: cada centroide al promedio de los puntos que le tocaron (con la asignación fija, el promedio es el punto que minimiza la suma de distancias al cuadrado).

Detenerse cuando una pasada no cambie ninguna asignación.

Como los dos pasos dejan la inercia igual o más pequeña y hay una cantidad finita de asignaciones posibles, el algoritmo siempre termina. Lo que no garantiza es terminar en el mínimo global: termina en un mínimo local, y cuál depende de dónde partió.

Un ejemplo trabajado a mano

Estos son los seis puntos que devuelve `util.datos_juguete()`, en el orden en que vienen (los grupos se numeran desde 0, igual que en el código):

$$\mathbf{x}_0 = (0,0), \quad \mathbf{x}_1 = (1,0), \quad \mathbf{x}_2 = (0,1), \quad \mathbf{x}_3 = (5,5), \quad \mathbf{x}_4 = (6,5), \quad \mathbf{x}_5 = (5,6).$$

Corramos k-means con $k = 2$ desde los centroides iniciales $\mathbf{c}_0 = (0,0)$ y $\mathbf{c}_1 = (1,0)$, que están los dos metidos en el grupo de abajo. La tabla lista la distancia al cuadrado de cada punto a cada centroide y la asignación que resulta.

punto	$\|\mathbf{x} - \mathbf{c}_0\|^2$	$\|\mathbf{x} - \mathbf{c}_1\|^2$	asignado a
$(0,0)$	0	1	0
$(1,0)$	1	0	1
$(0,1)$	1	2	0
$(5,5)$	50	41	1
$(6,5)$	61	50	1
$(5,6)$	61	52	1

La asignación es $(0,1,0,1,1,1)$ y su inercia vale $0+0+1+41+50+52 = 144$. Ahora el paso de actualización promedia cada grupo:

$$\mathbf{c}_0 = \tfrac{1}{2}[(0,0)+(0,1)] = (0,\ 0{,}5), \qquad \mathbf{c}_1 = \tfrac{1}{4}[(1,0)+(5,5)+(6,5)+(5,6)] = (4{,}25,\ 4).$$

Con los mismos puntos asignados, la inercia bajó de 144 a $37{,}25$: mover los centroides al promedio nunca la sube. La segunda iteración vuelve a asignar, y ahora $(1,0)$ cambia de bando (le queda más cerca $(0;\ 0{,}5)$ que $(4{,}25;\ 4)$):

$$\text{asignación } (0,0,0,1,1,1), \qquad \text{inercia } 11{,}9375,$$

y el promedio de cada trío da $\mathbf{c}_0 = (\tfrac{1}{3}, \tfrac{1}{3})$ y $\mathbf{c}_1 = (\tfrac{16}{3}, \tfrac{16}{3})$, con inercia $8/3 \approx 2{,}667$. La tercera iteración asigna de nuevo, obtiene otra vez $(0,0,0,1,1,1)$, y ahí el algoritmo se detiene: si la asignación no cambió, los promedios tampoco van a cambiar y la iteración siguiente sería idéntica.

Dos cosas que conviene retener de este ejemplo, porque el corrector las comprueba. La primera: la iteración en que se detecta la convergencia **también se cuenta**, así que el resultado son **tres** iteraciones, no dos. La segunda: la inercia bajó en cada paso ($144 \to 37{,}25 \to 11{,}9375 \to 2{,}667$), y esa sucesión decreciente es todo el argumento de por qué el algoritmo termina.

Evaluar sin etiquetas

Un agrupamiento no tiene «respuesta correcta» que comparar directamente, entre otras cosas porque los números con que se nombran los grupos son arbitrarios: la partición $(0,0,0,1,1,1)$ y la partición $(7,7,7,2,2,2)$ son la misma. El **índice de Rand** resuelve eso mirando pares en vez de elementos. Para cada uno de los $\binom{n}{2}$ pares se pregunta si las dos particiones lo tratan igual, es decir si las dos lo dejan junto o las dos lo separan; el índice es la proporción de pares en que hay acuerdo. Vale 1 exactamente cuando las dos particiones coinciden salvo el nombre de los grupos.

PCA

PCA busca las direcciones en que los datos varían más. Se centran los datos restando la media de cada columna, se calcula la matriz de covarianza muestral $S = X_c^\top X_c/(n-1)$, de forma (d,d), y se obtienen sus autovectores y autovalores. Como S es simétrica, los autovectores forman una base ortonormal y los autovalores son reales y no negativos. Ordenados de mayor a menor autovalor, el

autovector j es la dirección j de máxima varianza y su autovalor es exactamente la varianza de los datos proyectados sobre ella. Quedarse con las primeras m direcciones y descartar el resto entrega la mejor aproximación de rango m que existe, y lo que se pierde es, ni más ni menos, la varianza de las componentes descartadas.

Densidad

k-means describe cada grupo con un punto, así que solo puede encontrar grupos más o menos redondos y del mismo tamaño, y todo punto queda en alguno: no existe la categoría «esto es ruido». DBSCAN cambia el supuesto. Un grupo pasa a ser una **región densa**: se fija un radio `eps` y un mínimo de vecinos `min_muestras`, se declara **de núcleo** a todo punto que tenga al menos ese mínimo de vecinos dentro del radio (contándose a sí mismo), y un grupo es un conjunto de puntos de núcleo encadenados, más los puntos no densos que caen en el radio de alguno de ellos, que se llaman **de borde**. Lo que no alcanza a entrar en ninguna cadena es **ruido**. El algoritmo no necesita que le digan cuántos grupos hay, pero sí necesita que todos los grupos tengan una densidad parecida, porque el radio es uno solo.

Archivos

- `submission.py`: el único archivo que debes modificar.
- `util.py`: los conjuntos de datos, todos generados localmente con semilla fija. Ya está completo y no se toca.
- `grader.py`: corrector automático. Corre sin conexión a internet con `python grader.py`.
- `requirements.txt`: `numpy`, y nada más.

Está prohibido usar `scikit-learn` (o cualquier otra biblioteca de aprendizaje) en las funciones evaluadas: la tarea consiste justamente en escribir los algoritmos. El corrector revisa las importaciones del archivo.

Qué entregar

- **Entrega**: un solo archivo, `submission.py`, con las once funciones completadas. No se entrega nada más (ni informe, ni gráficos, ni un `.zip`) y no cambies las firmas ni el nombre del archivo, porque el corrector las llama tal como están escritas.
- Antes de entregar, ejecuta `python grader.py` en la carpeta de la tarea: debe reportar el puntaje completo de las pruebas visibles, que suman 60 de los 100 puntos. Llegar a ese total es condición *necesaria y no suficiente*, porque la nota la pone el corrector del curso, que corre esas mismas pruebas más otras sobre las mismas once funciones, con los mismos criterios.
- Puntaje total: 100 puntos, repartidos en las seis partes que vienen a continuación. Los 100 se corrigen solos, sin revisión a mano.
- No incluyas la carpeta `__pycache__`.

Problema 1: Agrupar con k-means **[36 puntos]**

Todo este problema va en `submission.py`. Las convenciones están en el docstring del archivo: `X` de forma (n, d) con una fila por observación, `C` de forma (k, d) con una fila por centroide, y una partición es un arreglo de enteros de forma $(n,)$ con los grupos numerados desde 0.

(a) k-means completo [24 puntos]

Implementa las cuatro funciones del algoritmo:

- `asignar_grupos(X, C)`: el índice del centroide más cercano a cada fila. Ante un empate exacto gana el centroide de índice menor.
- `actualizar_centroides(X, etiquetas, k, C_previo)`: el promedio de cada grupo. Si un grupo quedó **vacío**, su centroide es el de `C_previo`: el promedio de cero puntos no existe, y devolver `NaN` contamina todo lo que viene después.
- `inercia(X, C, etiquetas)`: la suma de distancias al cuadrado de cada punto a su centroide asignado.
- `kmeans(X, k, C0, max_iter=100)`: el bucle completo, con el criterio de parada del ejemplo trabajado más arriba y el diccionario de cinco claves que describe el docstring.

Pista

Para las distancias no hace falta ningún bucle en Python: la expresión `((X[:, None, :] - C[None, :, :]) ** 2).sum(axis=2)` arma la matriz (n, k) de distancias al cuadrado de una vez. Y no calcules la raíz cuadrada: para saber cuál centroide está más cerca basta comparar los cuadrados, que es lo mismo y es más barato. Ojo con el conteo de iteraciones: la que detecta la convergencia también cuenta.

(b) Inicialización k-means++ [12 puntos]

Implementa `inicializar_kmeanspp(X, k, semilla)`. El primer centroide se sortea uniformemente entre las filas de `X`; cada uno de los siguientes se sortea entre las filas con probabilidad proporcional a $D(\mathbf{x})^2$, la distancia al cuadrado de $\mathbf{x}$ al más cercano de los centroides ya elegidos. Un punto que quedó lejos de todo tiene mucha probabilidad de ser el próximo centroide, y uno que ya está bien cubierto casi ninguna. El docstring fija la secuencia exacta de sorteos, que es lo que hace reproducible el resultado.

Pista

El conjunto `util.datos_cinco_grupos()` tiene un grupo de 240 puntos y cuatro de 20. Con el sorteo uniforme, la probabilidad de que los cinco centroides caigan en cinco grupos distintos es minúscula: casi siempre tres o cuatro caen dentro del grupo grande y algún grupo pequeño se queda sin ninguno, que es exactamente el óptimo local que k-means no sabe abandonar. Ahí se nota k-means++.

Problema 2: Comparar dos particiones **[14 puntos]**

(a) El índice de Rand [14 puntos]

Implementa `indice_rand(a, b)`, la proporción de los $\binom{n}{2}$ pares en que las dos particiones están de acuerdo: los dos las dejan juntas o los dos las separan. Devuelve un `float` entre 0 y 1, y lanza `ValueError` si las particiones tienen largos distintos o si hay menos de dos elementos, porque sin al menos un par la proporción no está definida.

Pista

No hace falta recorrer los pares uno por uno. La matriz booleana `a[:, None] == a[None, :]` dice, en la posición (i, j), si la partición `a` deja juntos a i y j. Compara esa matriz con la de `b` y cuenta los aciertos del triángulo estrictamente superior, que es donde vive cada par exactamente una vez. Un control de escritorio: sobre las dos particiones de `util.particiones_ejemplo()` el resultado es $10/15 = 0{,}6667$.

Problema 3: Reducir dimensiones con PCA [22 puntos]

(a) PCA, proyección y reconstrucción [22 puntos]

Implementa `pca(X)`, `proyectar(X, modelo, m)` y `reconstruir(Z, modelo)`.

`pca` centra los datos, arma la covarianza muestral (dividiendo por $n - 1$), obtiene autovalores y autovectores con `np.linalg.eigh` y los ordena de mayor a menor autovalor. Como $-\mathbf{v}$ es tan autovector como $\mathbf{v}$, hace falta una convención de signo o el resultado no es reproducible: la coordenada de mayor valor absoluto de cada componente queda positiva. Devuelve el diccionario de cuatro claves que describe el docstring, con las componentes **por filas**. Lanza `ValueError` con menos de dos filas o con varianza total cero.

`proyectar` centra y multiplica por las primeras m componentes; `reconstruir` deshace esa operación y vuelve a sumar la media.

Pista

`np.linalg.eigh` devuelve los autovalores en orden *ascendente* y los autovectores en **columnas**. O sea que hay que dar vuelta el orden y transponer: los dos errores clásicos de esta parte se detectan solos si compruebas que `componentes @ componentes.T` da la identidad y que la varianza de la columna j de la proyección coincide con el autovalor j. El corrector comprueba exactamente eso, más la igualdad entre el error relativo de la reconstrucción de rango m y la varianza explicada que se descartó.

Problema 4: Densidad, y qué método elegir [28 puntos]

(a) DBSCAN [22 puntos]

Implementa `dbscan(X, eps, min_muestras)`. La vecindad de un punto son los puntos a distancia **euclidiana** menor o igual que `eps`, incluido él mismo; es de núcleo si su vecindad tiene al menos `min_muestras` puntos. El recorrido está fijado en el docstring y el corrector lo comprueba: se visitan los puntos por índice creciente, cada punto de núcleo que todavía no pertenece a ningún grupo abre uno nuevo, el grupo crece con una cola FIFO, y un punto de borde alcanzado desde dos grupos se queda en el primero que lo alcanzó. El ruido se marca con -1.

Control de escritorio. Sobre los ocho puntos alineados

$$(0,0),\ (0{,}5,0),\ (1,0),\ (1{,}5,0),\ (5,0),\ (8,0),\ (8{,}5,0),\ (9,0)$$

con `eps` $= 0{,}6$ y `min_muestras` $= 3$, el resultado es $(0, 0, 0, 0, -1, 1, 1, 1)$: dos grupos de cuatro y de tres, y el punto en $x = 5$ como ruido. Fíjate en que el primer punto y el cuarto *no* son de núcleo (solo tienen dos vecinos), pero igual entran al grupo 0 como puntos de borde.

(b) Elegir el método para un caso [6 puntos]

Implementa `elegir_metodo(caso)` para los dos casos que describe el docstring: `"lunas"` (dos grupos alargados y entrelazados, de densidad pareja) y `"gaussianos"` (tres nubes redondas de densidades muy distintas). Devuelve el método elegido y sus parámetros.

El corrector no se conforma con el nombre: corre el método que elegiste, con tus parámetros, sobre el conjunto correspondiente, y exige que la partición resultante tenga la cantidad correcta de grupos, poco ruido y un índice de Rand sobre $0{,}95$ contra las etiquetas verdaderas. Los parámetros tienen que servir de verdad, así que conviene probarlos antes en el intérprete.

Pista

Para elegir `eps` sirve mirar a qué distancia está el vecino número `min_muestras` de cada punto: un radio que deje a casi todos los puntos por sobre ese mínimo conecta el grupo sin salirse de él. Y recuerda que las dos lunas están separadas por menos de lo que mide cada luna: un radio demasiado grande las fusiona en un solo grupo.